Design and Verification of a Network-on-Chip (NoC) Using SystemVerilog and Universal Verification Methodology (UVM)

VLSI Design

Table of Content

1. Introduction

1.1 Purpose of the Study

1.2 Scope and Design Goals

2. RTL Details

2.1 Data Structure

2.2 Transmitter

2.3 Receiver

2.4 Router

2.5 Topology Example

3. UVM Verification

3.1 Environment Overview

3.2 Sequence Item

3.3 Sequencer

3.4 Scoreboard

3.5 Base Sequence

3.6 Integration in Test

4. Conclusion

I. Introduction

1.1 Purpose of Study

The purpose of this study is to design and verify a simplified Network-on-Chip (NoC) system using SystemVerilog and Universal Verification Methodology (UVM). The project aims to provide a comprehensive, hands-on exploration of interconnect architecture and modern verification techniques. The project serves as a practical learning platform for understanding how distributed nodes in a system-on-chip communicate efficiently using packet-based protocols, and how such systems are verified in a structured manner.

1.2 Scope and Goals

This project aims to design and verify a simplified Network-on-Chip (NoC) architecture using Universal Verification Methodology (UVM). The work is divided into two primary phases:
(1) hardware design of the NoC, and
(2) development of a reusable UVM-based verification environment.

Part 1: NoC Design

The objective is to construct a request–response Network-on-Chip interconnect where each node communicates through two virtual channels:

- Channel 0 (Request): 48-bit payload per packet (1 flit)
- Channel 1 (Response): 144-bit payload per packet (3 flits)

Each packet contains a 16-bit header, and all flits are 64 bits wide, defining the link width of the network. The design treats channels as virtual, meaning there are no physically separate links per channel—only dedicated I/O ports at each node. The links between nodes are full-duplex, supporting simultaneous 64-bit data transmission in both directions. Additionally, the network supports multicasting for broadcasting data to multiple destinations.
The primary hardware modules to implement include:

- Router: incorporating arbitration, virtual output queuing (VOQ), and routing control.
- Transmitter: handling packetization and data injection into the network.
- Receiver: responsible for packet assembly and delivery to the node interface.

Each node operates on its own clock domain, independent of the network clock, introducing clock domain crossing (CDC) considerations in design and verification.

Part 2: UVM-Based Verification

The second phase focuses on building a comprehensive UVM testbench to verify the NoC functionality. Core UVM components to develop include:
Driver, Monitor, Sequencer, Agent, Scoreboard, and Environment (Env).
A base sequence and randomized test will be implemented, where each node transmits packets with randomized content and destinations. The verification environment ensures that all packets are accurately delivered and reconstructed, validating the correctness, reliability, and synchronization of the network.

II. RTL Details

2.1 Data Structure

The Network-on-Chip employs a flit-based packet transmission scheme in which all data and control information are encapsulated within fixed-size 64-bit flits. Each flit contains a header that defines routing and control attributes essential for proper delivery and packet reconstruction. The packet header uses the same format as the flit header, with minor differences in control bits.

Field	Size	Description
source	4 bits	Identifier of the originating node.
multicast map	8 bits	Bitmask representing target nodes. A bit is set (1) if the corresponding node should receive the packet. Enables one-to-many data transmission.
vc	2 bits	Specifies the virtual channel index
first	1 bit	Set to 1 if this flit is the first flit of a packet.
last	1 bit	Set to 1 if this flit is the final flit of a packet.

Table 2.1 Flit Header Format

Each flit is therefore structured as:
[Source (4) | Multicast Map (8) | VC (2) | First (1) | Last (1) | Payload (48)]

The packet header shares the same structure as the flit header, with the First and Last bits unused (set as padding). This ensures consistent formatting across all transmitted units, simplifying encoding and decoding logic.
The network employs deterministic routing, meaning that flits originating from the same node and virtual channel follow a fixed, predefined path. As a result:
Flit order is preserved — flits from the same source and virtual channel arrive sequentially without reordering.
The receiver does not require explicit packet identifiers, as it assumes that a new packet begins only after the last flit of the previous one is received.

2.2 Transmitter

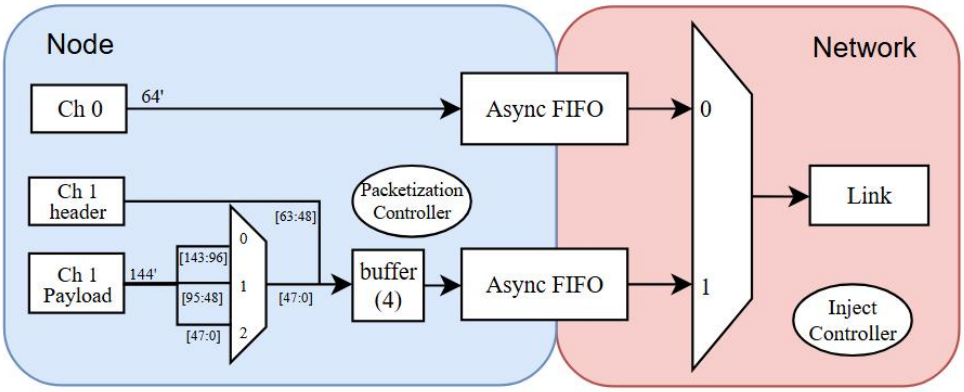

Figure 2.1 Transmitter Architecture of a Network-on-Chip Node

The Transmitter is responsible for preparing and injecting packets from each node into the Network-on-Chip. It serves as the interface between node-level data generation and the network fabric, ensuring correct packetization, buffering, synchronization, and injection control. The following explanation corresponds to the provided architectural diagram.

1. Functional Overview

Each node features two logical communication channels—Channel 0 for requests and Channel 1 for responses—both sharing the same physical network link through virtual channel multiplexing. The transmitter's role is to assemble packet data, manage timing differences between node and network clocks, and reliably forward data into the network through asynchronous FIFOs.

2. Channel Structure and Data Composition

Channel 0: Transmits 64-bit packets directly, as each packet fits within one flit.
Channel 1: Handles larger packets containing a 16-bit header and a 144-bit payload. The payload is divided into three 48-bit segments and combined with the header to form multiple 64-bit flits.
A Packetization Controller coordinates this division, merging the header and segmented payload data into proper flit sequences. Each flit is stored temporarily in a 4-entry buffer, allowing for flow control and arbitration before injection.

3. Clock Domain Crossing

Because each node operates on its own clock domain, asynchronous FIFOs are implemented to safely transfer data into the network's clock domain. These FIFOs provide synchronization, elasticity, and rate matching, ensuring data integrity under differing clock frequencies.
Async FIFO 0: For Channel 0 packets.
Async FIFO 1: For Channel 1 packets.

Each FIFO outputs into a multiplexer controlled by the Inject Controller, which selects the active channel for injection based on availability and network arbitration status.

4. Injection Control and Network Interface

The Inject Controller manages access to the shared physical network link. It monitors FIFO readiness and network availability, asserting control signals to launch packet data onto the full-duplex 64-bit link. This design allows concurrent data transmission in both directions, optimizing throughput and minimizing latency.

2.3 Receiver

The Receiver module is responsible for collecting incoming flits from the network, reconstructing complete packets, and delivering them to the corresponding node channels. It ensures synchronization between the network and node clock domains and guarantees correct packet boundary recognition through header control signals.

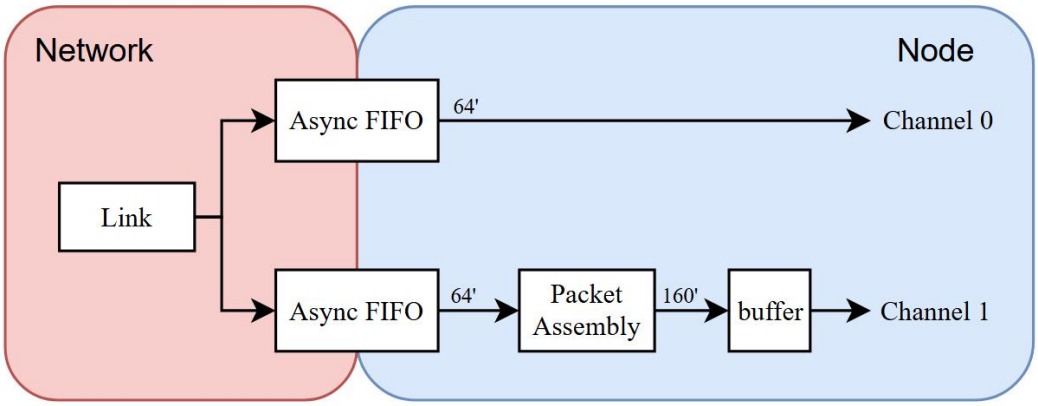

Figure 2.2 Receiver Architecture of a Network-on-Chip Node

1. Functional Overview

Each node receives incoming flits via a full-duplex link from the network. Similar to the transmitter, two asynchronous FIFOs are used to handle clock domain crossing (CDC) between the network clock and the local node clock.

FIFO 0: Handles flits for Channel 0.

FIFO 1: Handles flits for Channel 1.

Flits are dequeued from the FIFOs and forwarded to either the node directly (for Channel 0) or to the Packet Assembly unit (for Channel 1).

2. Packet Assembly Mechanism

The Packet Assembly unit reconstructs complete packets from a sequence of flits based on header flags (first and last). Each assembled packet entry contains:

Field	Size (bits)	Description
Header	16	Copied directly from the first flit received (first = 1).
Payload	144	Accumulated from subsequent flits until the last one is received (last = 1).

Table 2.1 Packet Assembler Entry

The receiver maintains an internal table of active packet entries, where each entry corresponds to a unique source node. Because deterministic routing guarantees in-order delivery from any given source and virtual channel, each source can have at most one active, incomplete packet at a time. Therefore, the source field in the flit header naturally serves as the unique tag identifying the corresponding assembly entry.

3. Assembly Procedure

(1) Start of Packet:

When a flit arrives with first = 1, a new packet entry is allocated.

The 16-bit header field is extracted and stored.

The source ID becomes the unique tag for tracking subsequent flits.

(2) Payload Accumulation:

Subsequent flits with matching source ID and first = 0 are concatenated into the 144-bit payload buffer.

Flit ordering ensures correct payload reconstruction without explicit packet IDs.
(3) Completion and Delivery:
When a flit with last = 1 is detected, the packet is marked ready for output.
The completed 160-bit packet (16-bit header + 144-bit payload) is pushed into the output buffer, which interfaces with Channel 1 of the node.

2.4 Router

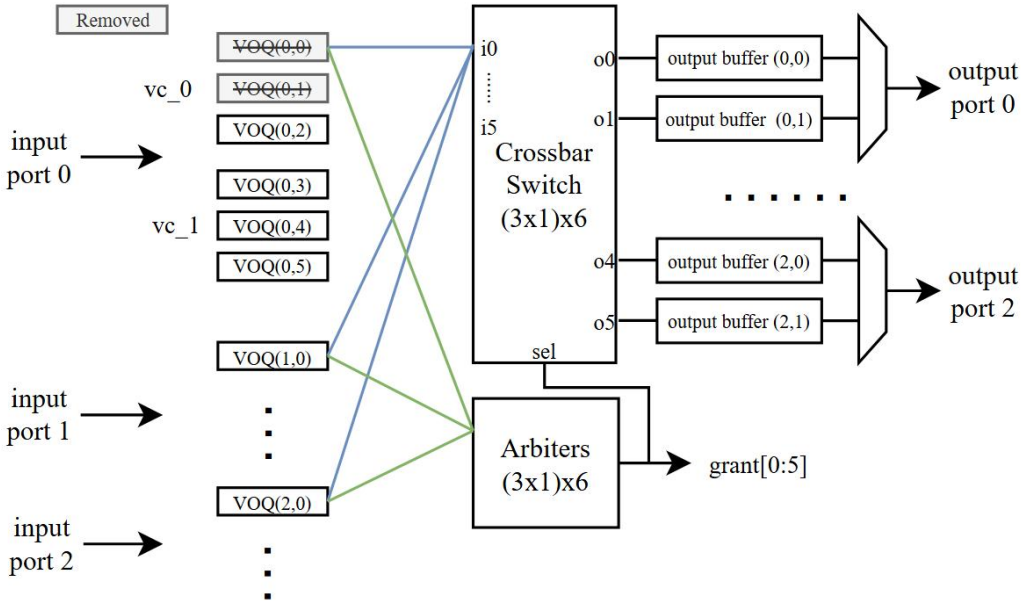

Figure 2.3: 3-port Router Architecture. Some VOQs are Removed/Disabled Due to Loopback Restriction.

The Router serves as the central switching element within the Network-on-Chip (NoC), responsible for directing packets from multiple input ports to multiple output ports based on their destination and virtual channel information. Its design combines VOQ, crossbar-based switching, and arbitration control to achieve high throughput and fairness across concurrent traffic flows. This section uses a 3-port router for example.

1. Overview
Each router connects several input ports to multiple output ports through a crossbar switch, coordinated by an arbiter array. Two virtual channels—vc0 and vc1—are supported per input port, corresponding respectively to the request and response channels in the system.
Each input port maintains a set of VOQs, where each queue corresponds to a distinct output destination. This eliminates head-of-line blocking and allows independent queuing per destination.

Input Ports: Three input ports (0-2), each receiving flits from neighboring nodes or local transmitters.
Virtual Channels (vc0, vc1): Logical separation of traffic classes (e.g., request vs. response).
Output Ports: Six output ports (0 – 5), each driving a connected node or network link.

2. Virtual Output Queues (VOQ)

Each input port implements multiple VOQs, one per output port. The VOQs store flits destined for each output independently, ensuring that a blocked packet on one output does not stall others.
For example:
VOQ(0,2) stores packets from input port 0 destined for output port 2.
VOQ(0,5) stores packets from input port 0 destined for output port 5.

This configuration results in a total of (number of input ports * number of output ports * number of VCs) queues.

3. Arbitration

At every clock cycle, arbiters determine which input port's flit may advance to each output port. The arbitration network is organized as a (3*1)*6 matrix, meaning each of the six output ports has a 3-input arbiter deciding among requests from the three input ports.
The arbitration process follows these general steps:
Each VOQ signals a request to its corresponding output port when it has data ready.
The arbiter associated with that output port evaluates all requests and issues one grant (encoded in grant[0:5]) based on a selected policy—typically round-robin or priority-based arbitration.
The granted flit is routed through the crossbar switch to the designated output port.

This design uses a round-robin arbiter following the design shown in Reference [1].

4. Routing Control

The router uses a static deterministic routing policy, where each output port is associated with a predefined forwarding mask.
Each router maintains a forwarding table, containing one mask entry per output port. Each mask indicates the set of destination nodes reachable through that port. For example, for a router with three output ports, the table might be:

Output Port	Forwarding Mask
0	0001
1	0010
2	1100

Table 2.2. Example Forwarding Mask Table

Each bit in the mask corresponds to a network node, where 1 indicates that packets destined for that node should be forwarded through this port.

5. Multicast Handling

This mechanism naturally supports multicast communication. A single incoming flit may be accepted by multiple VOQs corresponding to different output ports if their forwarding masks overlap with the flit's multicast map. Thus, flit replication occurs only inside routers, preventing redundant packet generation at the source node.

For example, using the forwarding table above:
If a packet targets Node 2 and Node 3 (multicast map = 1100),
The router identifies that both destinations are covered by port 2's mask (1100),
Therefore, the flit is forwarded only through output port 2, ensuring efficient one-hop multicasting without duplication.

6. Loop Prevention and Forwarding Rules
To avoid infinite routing loops or redundant transmissions:
Loopback forwarding is disallowed - a flit cannot be sent out through the same port from which it arrived.
Each router applies a port-based exclusion rule, ensuring that a flit is never reinserted into its incoming path.
This design provides an inherently loop-free routing fabric, simplifying network control while maintaining delivery correctness.
Thus the actual number of VOQs after this restriction becomes (number of input ports-1) * number of output ports * number of VCs.

2.5 Topology Example

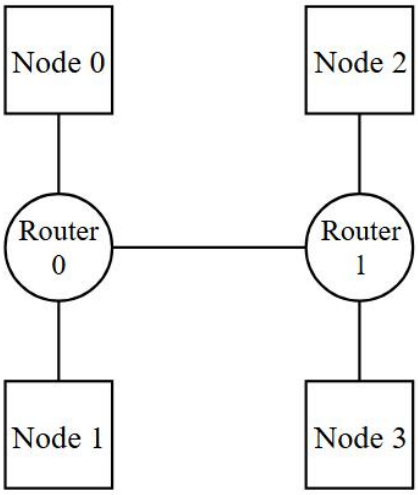

Figure 2.4: Example Network Topology (Top-Level System for UVM Verification)

Figure 2.4 illustrates a representative 4-node, 2-router Network-on-Chip topology, serving both as a conceptual example and as the top-level design for UVM-based verification.
The network consists of two routers (Router 0 and Router 1) interconnected by a bidirectional link.
Each router connects to two local nodes, forming a compact and symmetric structure:
Router 0 connects to Node 0 and Node 1.
Router 1 connects to Node 2 and Node 3.
Each node communicates through its local router to reach other nodes in the network.

This configuration represents a small-scale hierarchical NoC and serves as the testbench integration model for UVM verification.

III. UVM Verification

3.1. Overview

The verification of the NoC design is performed using the Universal Verification Methodology (UVM) framework, emphasizing modularity, scalability, and reusability. Since all nodes in the network are architecturally identical, a single standardized agent and interface structure is instantiated for each node, ensuring uniform testbench configuration and simplified maintenance.

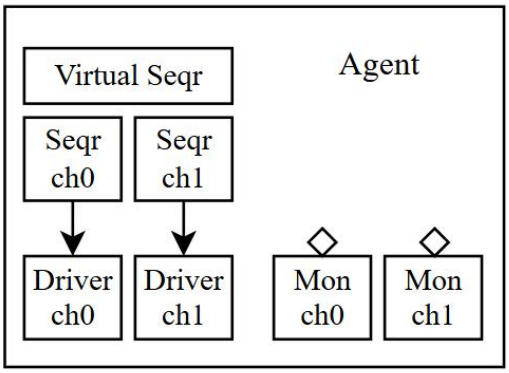

Figure 3.1 UVM Agent Structure for Dual-Channel Node Interface

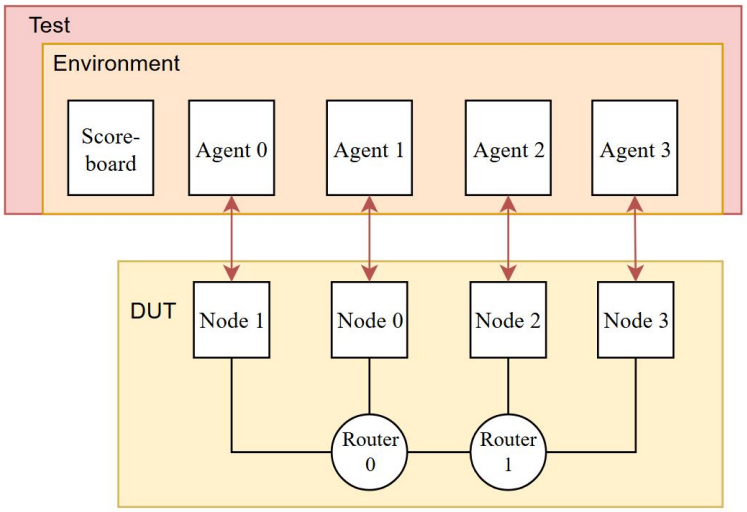

Figure 3.2 UVM Testbench Architecture for the NoC System

1. Agent-Level Structure

Each UVM agent encapsulates the verification components required to drive and monitor a node's dual-channel communication interface. As illustrated in Figure 6, each agent comprises:

- Virtual Sequencer (Virtual Seqr):
  Coordinates stimulus generation across both channels, enabling synchronized or independent transaction sequences.
- Channel Sequencers (Seqr ch0 / Seqr ch1):
  Manage sequence execution for each respective channel. Channel 0 handles request transactions, and Channel 1 handles response transactions.
- Drivers (Driver ch0 / Driver ch1):

Convert high-level transaction objects into low-level signal activities at the interface, initiating packet transmission toward the Design Under Test (DUT).

- Monitors (Mon ch0 / Mon ch1):
  Passively observe signal-level behavior and convert observed transactions back into UVM objects. These are then forwarded to analysis components, such as the scoreboard, for functional checking.

This modular channel separation enables independent verification of request and response paths while supporting cross-channel coordination via the virtual sequencer.

2. Environment-Level Architecture

At the top level, the UVM environment (env) integrates all agents and the central checking component, as shown in Figure 3.2.

- Agents (Agent 0–3):
  Four identical agents correspond to the four nodes (Node 0-Node 3) of the DUT. Each agent interfaces with its associated node through a dedicated virtual interface, maintaining symmetry across the verification structure.
- Scoreboard:
  Collects observed transactions from all agents' monitors. It compares sent and received packets to ensure data integrity and correct routing across the network.
- DUT (Design Under Test):
  The DUT corresponds to the previously defined 4-node, 2-router network topology. The testbench communicates with the DUT exclusively through the node interfaces.
- Test Layer
  The top-level test class controls simulation execution, configures the environment, and launches virtual sequences that stimulate random packet generation across all nodes.

3. Functional Objective

The UVM verification environment validates:
Correct packet routing, multicasting, and delivery across the network.
Robust handling of clock domain crossing and asynchronous communication.
Integrity of transmitted data under random and concurrent node activity.

This modular UVM setup ensures that each verification component can be reused, extended, or parameterized for larger NoC configurations, enabling a scalable and standard verification workflow.

3.2 Sequence Item (noc_transaction Class)

1. Overview

The noc_transaction class defines the fundamental data object used for stimulus generation, monitoring, and scoreboard comparison within the UVM verification environment. Each instance of this class represents a single NoC transaction, encapsulating both header fields and payload data to

model a complete packet or flit according to the NoC protocol.

Field	Type/Size	Description
source	4 bits	Identifies the source node that generated the packet.
dest_map	8 bits	Multicast destination bitmask;
vc	2 bits	Virtual channel index
padding	2 bits	Reserved field for protocol alignment
rx_cid	Integer	Receiving node identifier used by the scoreboard for packet tracking.
imp_src	String	Source of the transaction within the testbench, e.g., transmitter or monitor.

Table 3.1 UVM Sequence Item Fields

2. Internal Constraints

To enforce correct packet size according to the virtual channel configuration, a constraint limits the effective payload length for ch_0 transactions:

```
constraint c_payload_size {
    if (vc != 2'b10)
        data[143:48] == 0; // Force unused bits to zero for 48-bit payloads
}
```

This ensures that packets using Channel 0 are encoded as single-flit packets (48-bit payload), while Channel 1 supports full three-flit (144-bit) payloads.

3.3 Sequencer

Two levels of sequencers are used: a channel sequencer for each communication channel and a virtual sequencer for cross-channel coordination within an agent.

Only the virtual sequencer is exposed to higher levels and a virtual sequence is used (see Section 3.5)

3.4 Scoreboard

1. Packet Information Structure

Each transmitted transaction is represented by a pkt_info_s structure, which stores metadata for end-to-end tracking:

```
typedef struct {
    bit [3:0]   source;  // Source node ID
    bit [7:0]   recv_map; // Destination confirmation bitmap (cleared upon reception)
    bit [143:0] data;    // Payload data for content matching
    bit         done;    // Flag indicating complete reception
    time        tx_time; // Transmission timestamp
    time        rx_time; // Final reception timestamp
} pkt_info_s;
```

2. Transmission Recording

When a transmit event occurs (tx_write()), a new pkt_info_s record is created and pushed into a queue corresponding to the sender's node and virtual channel.
These queues are organized as:

```
pkt_info_s sent_q[4][2][$];  // [node_id][vc][$]
```

Each node maintains two dynamic queues (one per virtual channel) that hold all unverified outgoing transactions.

The recv_map field of each record mirrors the intended destinations of the transmitted packet. Each bit corresponds to a recipient node. When a node successfully receives the packet, the corresponding bit is cleared.

The scoreboard later uses this map to confirm whether each targeted node has successfully received its respective copy.

3. Reception Matching
When a receive event occurs (rx_write()):
The scoreboard starts a search on the send_queue to look for a packet that matches the received packet.

4. Error Conditions

1. Invalid Packet VC
Condition: The received packet's virtual channel index does not match the monitor's channel identifier.
Effect: Indicates that a packet was delivered to the wrong channel monitor, suggesting a routing or arbitration fault.

2. Wrong Multicast Target
Condition: The receiving node ID (rx_cid) is not included in the packet's destination bitmask (dest_map).
Effect: The packet reached an unintended recipient, violating multicast routing correctness.

3. No Matching Transmission Record
Condition: The scoreboard cannot find a corresponding transmit entry in the sender's queue that matches the packet's source, vc, and data.
Effect: The received packet has no valid origin or has been corrupted/lost.

4. Completion Verification (Check Phase)
At the end of simulation (UVM check phase), the scoreboard performs a completeness check across all queues:
If any packet record in sent_q still contains active bits in its recv_map, it implies that not all destinations received the packet.
These are reported as missing deliveries, identifying the responsible node and the remaining

destination map.

Together, these checks provide comprehensive validation of the NoC's routing accuracy, multicast reliability, and end-to-end delivery integrity.

5. Functional Role

The scoreboard ensures that:

- All packets are delivered to their intended destinations, without duplication or loss.
- Payload integrity is maintained across transmissions.
- Multicast delivery correctness is verified through destination bit tracking.
- Latency consistency can be evaluated for network timing analysis.

3.5 Base Sequence and Virtual Sequence

The base sequence and its virtual counterpart define the stimulus generation strategy for the Network-on-Chip verification. They coordinate randomized packet generation across all nodes and virtual channels.

1. Base Sequence (noc_base_seq)

The noc_base_seq class is the fundamental transaction generator, producing randomized NoC packets for a single node and channel. It defines configurable parameters for the number of nodes, packet count, and transmission interval.

```
class noc_base_seq extends uvm_sequence#(noc_transaction);
    int unsigned num_nodes      = 4;   // Total nodes in network
    int unsigned tx_interval_min = 0;   // Minimum idle cycles between packets
    int unsigned tx_interval_max = 10;  // Maximum idle cycles between packets
    int unsigned total_packets   = 10;  // Number of packets per test
    int client_id;                     // Node ID for this sequencer
    bit [1:0] vc = 2'b00, pkt_vc;      // Virtual channel configuration
    const bit [1:0] def_vc_list = '{2'b01, 2'b10}; // Default VC list

    `uvm_object_utils(noc_base_seq)
    `uvm_declare_p_sequencer(client_sequencer)
```

2. Operation Flow

(1) Configuration Retrieval:

In pre_body(), parameters are read from the UVM configuration database. Defaults are retained if no external settings are provided.

(2) Packet Generation:

Within the body() task, packets are randomized and transmitted in a loop for the specified total_packets. Each packet's virtual channel (pkt_vc) is chosen randomly unless pre-assigned (vc should be pre-assigned to ensure correct behavior, otherwise it will raise "Invalid VC" Error).

(3) Randomization Rules:
The packet's source field must match the node's ID (client_id).
The destination bitmask must target at least one node.

(4) Transmission Timing:
After each packet is sent, a random number of idle cycles (tx_interval) is inserted between transmissions to emulate variable network load conditions.

This structure allows scalable and randomized traffic injection for both request and response channels, promoting thorough coverage of network behavior under dynamic load.

3. Channel-Specific Sequences
Derived classes such as noc_base_seq_ch0 and noc_base_seq_ch1 extend the base sequence, overriding the virtual channel index (vc) to bind the stimulus specifically to the appropriate channel.

4. Virtual Sequence (noc_base_vseq)
The noc_base_vseq class encapsulates base sequences for both channels using the virtual sequencer.

```
class noc_base_vseq extends uvm_sequence;
    `uvm_object_utils(noc_base_vseq)
    `uvm_declare_p_sequencer(client_vsequencer)
    noc_base_seq_ch0 seq_ch0;
    noc_base_seq_ch1 seq_ch1;
    task pre_body();
        seq_ch0 = noc_base_seq_ch0::type_id::create("seq_ch0");
        seq_ch1 = noc_base_seq_ch1::type_id::create("seq_ch1");
    endtask

    task body();
        fork
            seq_ch0.start(p_sequencer.sqr_ch0);
            seq_ch1.start(p_sequencer.sqr_ch1);
        join
    endtask
endclass
```

The virtual sequence performs the following:
Instantiates channel-specific sequences for both request and response paths.
Launches them concurrently using fork–join, enabling full-duplex traffic generation between nodes.
Delegates execution to the respective channel sequencers (sqr_ch0 and sqr_ch1) within the virtual sequencer.

3.6 Integration in Test

At the test layer, the virtual sequence (noc_base_vseq) is assigned to the virtual sequencer (client_vsequencer) and executed once per simulation.

```
virtual task run_phase(uvm_phase phase);
    phase.phase_done.set_drain_time(this, 1000ns);
    phase.raise_objection(this, "Test raise objection");

    fork
        vseq[0].start(env.agents[0].vsqr);
        vseq[1].start(env.agents[1].vsqr);
        vseq[2].start(env.agents[2].vsqr);
        vseq[3].start(env.agents[3].vsqr);
    join
    #100ns;
    phase.drop_objection(this);
endtask
```

Figure 3.3 shows a successful run of the base test.

```
** Report counts by id
UVM_FATAL :    0
UVM_ERROR :    0
UVM_WARNING :    0
UVM_INFO :   71
** Report counts by severity

--- UVM Report Summary ---
```

Figure 3.3 Simulation Result of Base Test

IV. Conclusion

This project demonstrates the design and verification of a basic Network-on-Chip (NoC) system using SystemVerilog and the Universal Verification Methodology.

On the design side, the NoC integrates essential components including transmitter, receiver, and router. The network implements flit-based, virtual-channel communication with deterministic routing and multicast support. The system also manages clock domain crossings, virtual output queuing, and loop-free routing, while maintaining simplicity and scalability suitable for larger SoC integration.

On the verification side, the UVM environment establishes a reusable and hierarchical testbench architecture. The scoreboard rigorously checks delivery correctness, data integrity, and multicast behavior. Controller randomized traffic sequences and timing ensure broad coverage and robustness under realistic network conditions.

Overall, this project provides a complete workflow covering both NoC architecture design and UVM-based functional verification. The project illustrates key practices in digital system design, protocol verification, and modular testbench development.

Reference

[1] Weber, Matt & Weber@ieee, Matthew & Org,. Arbiters: design ideas and coding styles. https://www.researchgate.net/publication/228693610_Arbiters_design_ideas_and_coding_styles